

AUTOMATED FORECASTING SYSTEMS

Auto-ARIMA · Auto Prophet · AutoML · AutoML + Feature Hashing

Shivam Musterya - 2025 Batch Alumni

AGENDA

01

Time Series Fundamentals

Components, patterns & data types

02

Auto-ARIMA

Automated ARIMA parameter selection

03

Prophet

Meta's flexible forecasting tool

04

AutoML for Forecasting

Machine learning automation pipeline

05

Hashing + AutoML

Scalable multi-series forecasting

06

Comparison & Use Cases

Which method to choose?

07

Demo & Exercises

Hands-on code examples

Why Forecasting Matters

Accurate predictions power decisions across every major industry

Amazon

~35%

of revenue driven by
ML-based demand
forecasting

Netflix

\$1B+

saved annually by
predicting viewing
behavior

Banks

30–50%

reduction in credit loss
with predictive risk
models

Hospitals

20%

reduction in patient wait
times via demand
forecasting

What is a Time Series?

A time series is a sequence of data points indexed in chronological order, collected at successive equally-spaced points in time.



Sales

Daily / weekly / monthly
revenue



Stock Prices

Intra-day & end-of-day
prices



Weather

Temperature, rainfall, wind



Health

Heart rate, glucose, steps

Key Properties:

Ordered — sequence matters **Temporal Dependency** — past influences future **Regularity** — fixed interval collection

Time Series Components

Trend



Long-term increase or decrease in the data. Captured by the overall direction of the series over time.

e.g. Rising annual sales, Population growth

Seasonality



Regular, predictable cycles repeated at fixed intervals — daily, weekly, yearly.

e.g. Holiday spikes, summer tourism peaks

Cyclicity



Irregular fluctuations over longer periods, not fixed frequency. Often tied to economic or business cycles.

e.g. Recession cycles, housing market waves

Noise



Random, irregular variation that cannot be attributed to the other components.

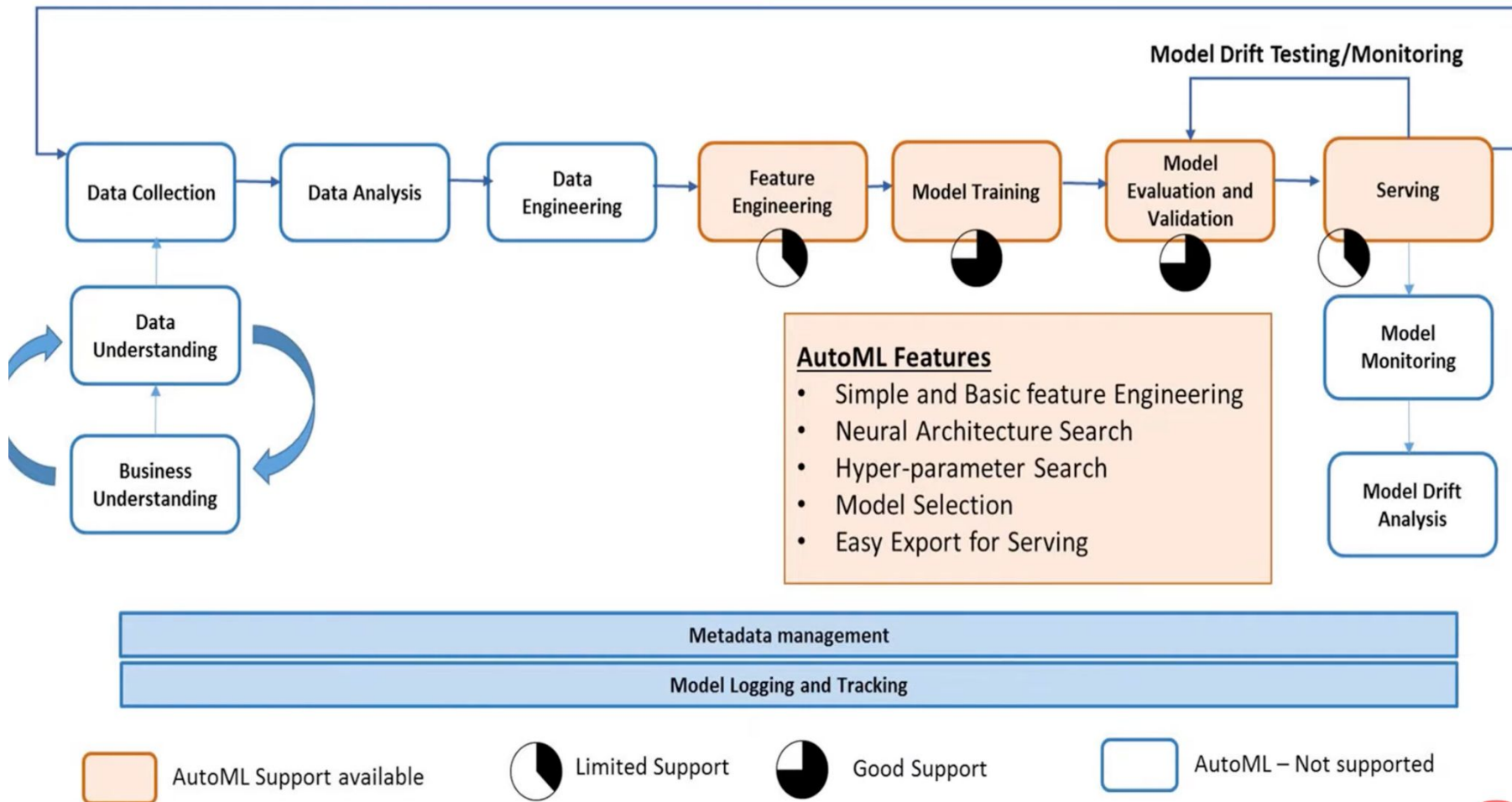
e.g. Unexpected events, measurement errors

Decomposition: $Y(t) = \text{Trend} + \text{Seasonality} + \text{Cyclicity} + \text{Noise}$

Traditional vs Automated Forecasting

Aspect	Traditional (Manual)	Automated
Model Selection	Expert manually tests models	Algorithmic search across models
Parameter Tuning	Grid search by hand (slow)	Automated AIC/BIC optimization
Scalability	One model per analyst	Hundreds of series simultaneously
Speed	Days to weeks	Minutes to hours
Expertise Required	High — statistician needed	Moderate — practitioner level
Reproducibility	Varies by analyst	Fully reproducible pipeline
Adaptability	Manual re-training	Automatic retraining triggers

Model Re-calibration



ARIMA: The Classic Forecasting Model

AR AutoRegressive

Uses past values of the variable to predict future values.
Correlation between observations.

$$X_t = \varphi_1 X_{t-1} + \varphi_2 X_{t-2} + \dots + \varepsilon_t$$

Formula

I Integrated

Makes the series stationary by differencing.
Removes trends and unit roots.

$$\Delta X_t = X_t - X_{t-1}$$

Formula

MA Moving Average

Uses past forecast errors in the model.
Smooths out short-term fluctuations.

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \dots$$

Formula

ARIMA(p, d, q) — Combined model: AR order p · Differencing d · MA order q

ARIMA Parameters: p , d , q

p

AR Order

Number of lag observations (lookback window).

- Identified via PACF plot cutoff.
- Too high: overfitting; Too low: misses patterns

Diagnostic: PACF (Partial Autocorrelation Function)

d

Differencing Order

Number of times to difference the series.

- $d=0$: already stationary
- $d=1$: removes linear trend
- $d=2$: removes quadratic trend

Diagnostic: ADF / KPSS stationarity test

q

MA Order

Number of lagged forecast errors.

- Identified via ACF plot cutoff.
- Controls smoothness of prediction.

Diagnostic: ACF (Autocorrelation Function)

Manual ARIMA: Challenges & Limitations

-  Expert knowledge required — must interpret ACF/PACF plots correctly
-  Time-consuming — testing each (p,d,q) combination manually is slow
-  Subjective — different analysts may choose different parameters
-  Not scalable — impractical for hundreds or thousands of time series
-  No automated retraining — must be manually updated when patterns change
-  Stationarity testing — requires manual ADF/KPSS interpretation
-  Seasonality ignored — basic ARIMA cannot handle seasonal patterns (needs SARIMA extension)
-  Cost — intensive analyst time = high operational cost for large-scale forecasting

Auto-ARIMA: Automated Parameter Selection

How Auto-ARIMA Works

1. Run stationarity test (ADF/KPSS) to determine d
2. Define search space for p and q (e.g. 0–5)
3. Fit all candidate ARIMA(p,d,q) models
4. Evaluate each using AIC or BIC criterion
5. Select model with lowest AIC/BIC score
6. Optionally test seasonal extensions (SARIMA)
7. Return best model + confidence intervals

Key Benefits

- ✓ No manual ACF/PACF reading
- ✓ Handles stationarity automatically
- ✓ Explores full search space
- ✓ Consistent, reproducible results
- ✓ Supports parallel fitting
- ✓ Works with pmdarima library

AIC: Akaike Information Criterion

$$\text{AIC} = 2k - 2 \ln(\hat{\mathcal{L}})$$

k = number of model parameters $\hat{\mathcal{L}}$ = maximized value of likelihood function

Penalizes Complexity

Adding more parameters always improves model fit. AIC adds a penalty term $2k$ to prevent overfitting — more complex models must substantially improve fit to be selected.

Lower is Better

The model with the lowest AIC score wins. Auto-ARIMA tries all combinations of (p,d,q) and picks the configuration with minimal AIC.

BIC Alternative

Bayesian Information Criterion (BIC = $k \cdot \ln(n) - 2 \cdot \ln(\hat{\mathcal{L}})$) applies a heavier penalty for parameters, favoring simpler models on large datasets.

SARIMA: Seasonal Extension of ARIMA

$\text{SARIMA}(p, d, q)(P, D, Q)_m$ where m = seasonal period

P

Seasonal AR order

Seasonal autoregressive lags at multiples of m

D

Seasonal Differencing

Seasonal difference to remove seasonal unit root

Q

Seasonal MA order

Seasonal moving average terms at multiples of m

m

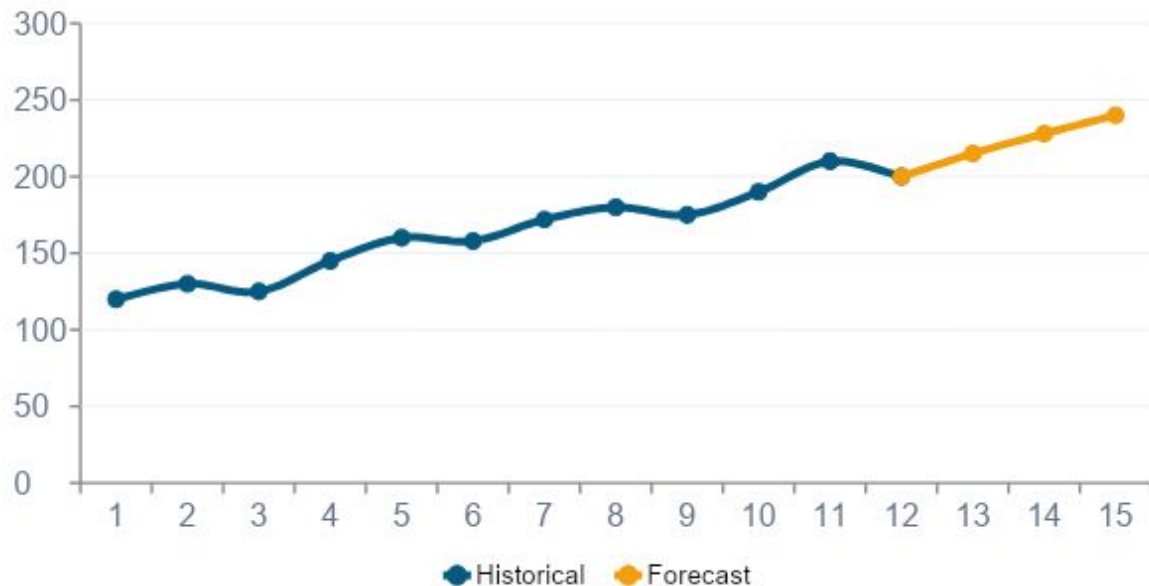
Seasonal Period

12 = monthly, 4 = quarterly, 7 = weekly, 24 = hourly

Example: $\text{SARIMA}(1,1,1)(1,1,1)_{12} \rightarrow$ Monthly data with yearly seasonality

Auto-ARIMA: Forecast Output Example

Monthly Sales — Actual vs Auto-ARIMA Forecast



Model Info

Best Model:

SARIMA(1,1,1)(1,0,0)₁₂

AIC Score: 284.3

RMSE (CV): 8.7

MAPE: 4.2%

Confidence: 95% CI

Horizon: 3 months

Prophet

Meta's open-source tool for forecasting time series with strong seasonal patterns

Prophet: Facebook's Forecasting Tool

Released by Meta (Facebook) in 2017 — designed for business analysts, not just statisticians



Handles Holidays

Manually specify custom holidays and events with variable windows



Multiple Seasonality

Daily, weekly, yearly — all handled simultaneously



Changepoints

Automatically detects trend changes without manual intervention



Fast & Scalable

Stan-based backend — runs in seconds for most datasets



Python & R

Simple API, minimal data preprocessing required



Uncertainty Quantification

Built-in confidence intervals using Monte Carlo simulation

The Bakery Analogy — Understanding Prophet

Think of Prophet like a bakery forecasting daily bread demand:

 **Trend $g(t)$**

 Overall sales growing as neighbourhood expands

→ *Piecewise linear or logistic growth with automatic changepoints*

 **Seasonality $s(t)$**

 More bread sold on weekends; spikes before Christmas

→ *Fourier series captures multiple overlapping seasonal cycles*

 **Holidays $h(t)$**

 Valentine's Day, Eid, Christmas cause special demand spikes

→ *User-specified holiday list with window around each date*

 **Noise $\epsilon(t)$**

 Unexpected weather, supplier shortage — unpredictable

→ *Modeled as Laplace noise in Bayesian framework*

Prophet's Decomposition Formula

$$y(t) = g(t) + s(t) + h(t) + \epsilon(t)$$

$g(t)$

Captures long-term non-periodic changes.

Two options:

- Linear: for unbounded growth
- Logistic: for growth with saturation limit (S-curve)

$s(t)$

Seasonality

Periodic effects modeled with Fourier series.

Can include:

- Weekly (period=7)
- Yearly (period=365.25)
- Custom periods

$h(t)$

Holiday Effects

Effect of known irregular events.

Each holiday gets its own coefficient.

Window parameter controls days before/after.

$\epsilon(t)$

Error / Noise

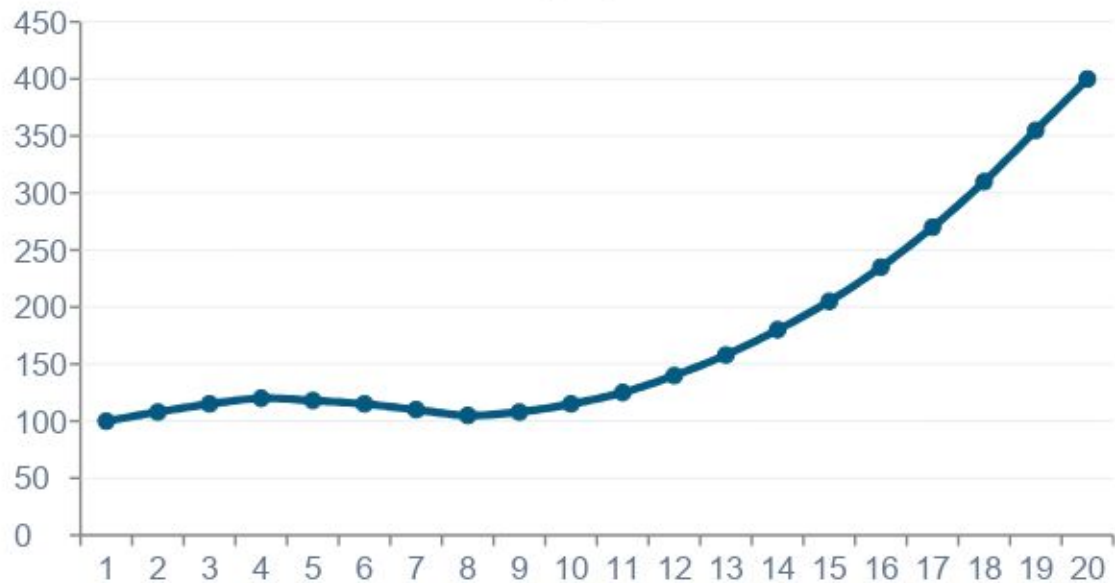
Unexplained variance.

Assumed to follow a Laplace distribution.

Prophet provides credible intervals for uncertainty.

Changepoint Detection in Prophet

Automatic Changepoint Detection



Key Settings

- `n_changepoints` (default=25)
- Number of potential change locations
- `changepoint_range` (default=0.8)
- Search in first 80% of data
- `changepoint_prior_scale`
- Controls flexibility — higher = more changepoints
- Manual override possible:
- `model.changepoints = ['2023-01-01']`

Additive vs Multiplicative Seasonality

Additive Seasonality

- Seasonal amplitude stays CONSTANT over time
- Formula: $y(t) = T + S + \epsilon$
- Use when: seasonal fluctuations don't scale with trend level
- Example: Temperature cycles — $\pm 5^{\circ}\text{C}$ regardless of base temp
- Prophet default: additive for yearly / weekly seasonality
- Fourier series: $s(t) = \sum [a \cos(2\pi nt/P) + b \sin(2\pi nt/P)]$

Multiplicative Seasonality

- Seasonal amplitude GROWS proportionally with trend
- Formula: $y(t) = T \times S \times \epsilon$
- Use when: higher baseline means larger seasonal swings
- Example: E-commerce sales — holiday spike is % of monthly base
- Set in Prophet: `model = Prophet(seasonality_mode='multiplicative')`
- Better for exponentially growing time series

Prophet Hyperparameters

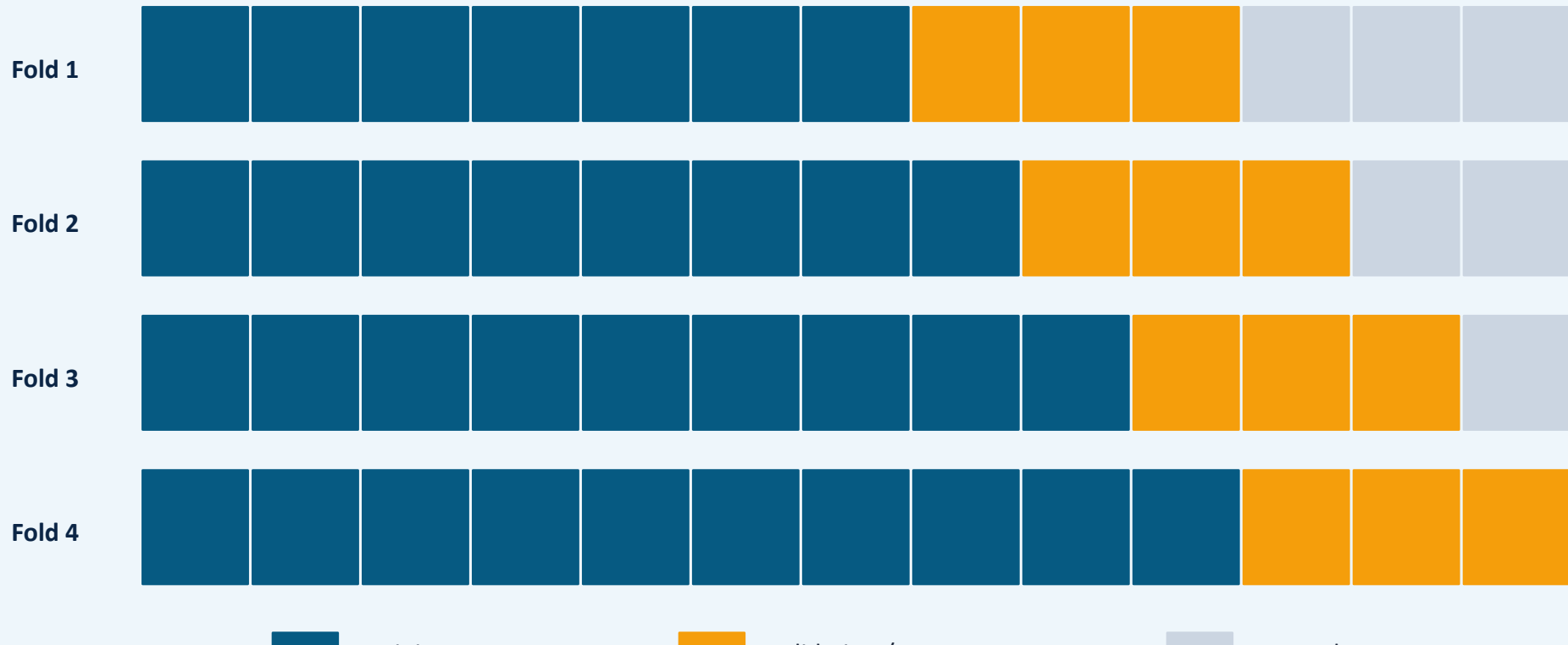
- `changepoint_prior_scale` (default=0.05) — Controls trend flexibility. Higher → more overfitting, Lower → underfitting
- `seasonality_prior_scale` (default=10) — Controls amplitude of seasonal terms. Higher → stronger seasonality
- `holidays_prior_scale` (default=10) — Flexibility for holiday effect estimates
- `seasonality_mode` — 'additive' (default) or 'multiplicative'
- `yearly_seasonality`, `weekly_seasonality`, `daily_seasonality` — True/False/integer (Fourier order)
- `n_changepoints` (default=25) — Number of potential trend change points to consider
- `changepoint_range` (default=0.8) — Proportion of history where changepoints are placed
- `interval_width` (default=0.8) — Width of confidence interval for uncertainty estimation

Automating Prophet: Grid Search + Cross Validation

- Define hyperparameter grid: `changepoint_prior_scale` × `seasonality_prior_scale` × `seasonality_mode`
- Use Prophet's built-in `cross_validation()` for walk-forward evaluation
- Compute performance metrics: `performance_metrics()` returns MAE, RMSE, MAPE per horizon
- Select best configuration based on average RMSE across all CV windows
- Parallelise with `joblib.Parallel` or `multiprocessing` for faster search
- Typical grid size: $3 \times 3 \times 2 = 18$ configurations, each evaluated across multiple windows
- Libraries: prophet, itertools.product, pandas, joblib
- Result: fully automated Prophet model selection with zero manual tuning

Walk-Forward Cross Validation

Traditional k-fold CV is invalid for time series — data must not leak future into past



AutoML for Forecasting

Applying machine learning automation to time series prediction pipelines

Why Use Machine Learning for Forecasting?

- Traditional statistical models (ARIMA/Prophet) assume linear relationships and specific data structures
- ML models can capture non-linear interactions, complex feature relationships, and multi-variate inputs
- Gradient Boosting (LightGBM, XGBoost) consistently outperforms ARIMA on many real-world datasets
- ML excels when exogenous variables matter — promotions, weather, economic indicators
- Handles large datasets efficiently — millions of rows without memory issues
- Supports ensembling — combine multiple models for better accuracy than any single model
- Feature importance: understand which lag features or covariates drive predictions
- Limitation: ML does NOT natively understand time — requires feature engineering to inject temporal structure

Converting Time Series to Tabular Format

ML models expect a row-per-prediction table. We create features from time series structure.

Input: Raw Time Series

Date	Sales
2024-01-01	120
2024-01-02	135
2024-01-03	128
2024-01-04	142
2024-01-05	155



Output: ML Feature Table

lag_1	lag_2	roll_3	dow	Target
120	—	—	3	135
135	120	—	4	128
128	135	128	5	142
142	128	130	6	155
155	142	142	0	167

lag_n = value n periods ago · roll_n = rolling mean n periods · dow = day of week

Feature Engineering for ML Forecasting

Lag Features

- `lag_1`, `lag_2`, ..., `lag_n`
- `lag_7` (weekly), `lag_365` (yearly)
- Captures autocorrelation in the series

Rolling Statistics

- Rolling mean (7d, 14d, 30d)
- Rolling std — measures volatility
- Rolling min/max, rolling median

Date/Time Features

- Day of week (0–6)
- Month, quarter, year
- Hour, week of year, `is_weekend`

Exogenous Variables

- Price, promotions, weather
- Economic indicators
- Competitor activity, holidays

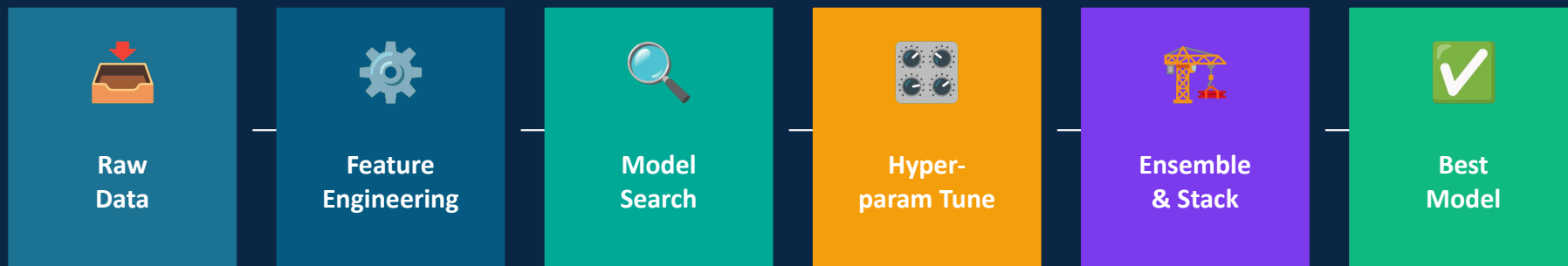
Target Encoding

- Mean target by category
- Smoothed category encodings
- Leave-one-out to prevent leakage

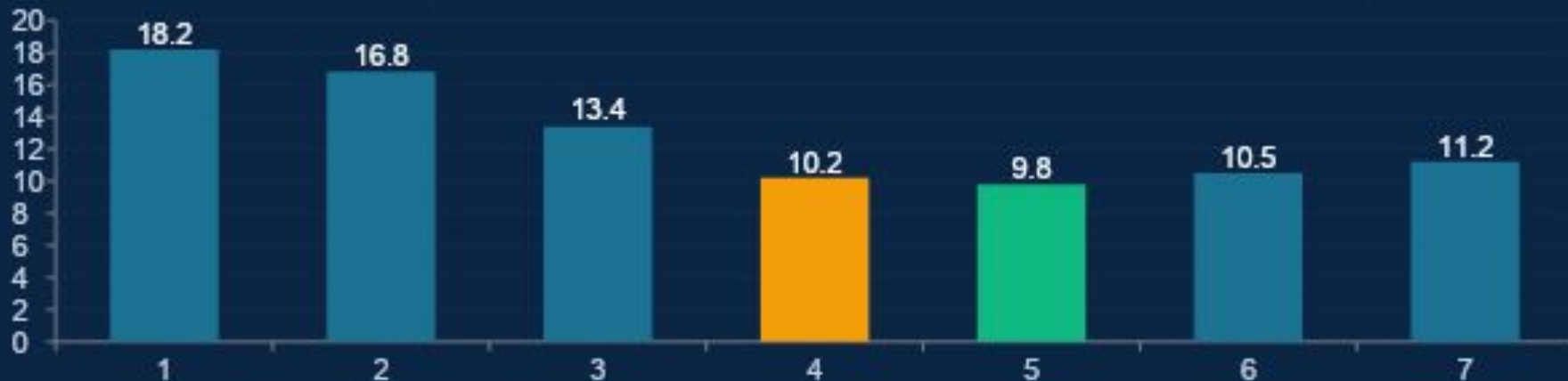
Interaction Features

- `lag` × `day_of_week`
- `rolling_mean` × `price`
- Seasonal × trend interaction

AutoML Pipeline for Forecasting



Model Comparison — Cross Validated RMSE (lower is better)



Hyperparameter Search Methods

Grid Search

✓ Exhaustive — covers all combinations

✗ Slow: $O(n^d)$ — exponential in dimensions

Best for: Small, well-defined search spaces

Complexity: $O(n^d)$

Random Search

✓ 2–4× faster, similar accuracy

✗ No learning — may miss optimal regions

Best for: Medium search spaces, limited budget

Complexity: $O(n)$

Bayesian Optimization

✓ Learns from past trials, efficient

✗ Sequential — harder to parallelize

Best for: Expensive models, many parameters

Complexity: $O(n \log n)$

Walk-Forward Validation for AutoML

- Standard k-fold cross-validation shuffles data — invalid for time series (future leaks into past)
- Walk-forward CV: expand training window forward in time, always predict future only
- Minimum initial training size: depends on data frequency and desired stability
- Step size: how many periods to advance each fold (1 = rolling, n = blocked)
- Gap period: add buffer between train and test to avoid leakage from autocorrelation
- Aggregate metrics: average MAE/RMSE across all folds — more reliable than single split
- TimeSeriesSplit (sklearn): implements walk-forward for any ML model
- Prophet: `cross_validation(model, initial='365 days', period='30 days', horizon='90 days')`

AutoML Forecasting Frameworks

FLAML

Fast, Lightweight AutoML by Microsoft.
Supports LightGBM, XGBoost, time series.
Efficient search with minimal tuning.

```
pip install flaml[ts]
```

AutoTS

Purpose-built for time series. Supports 20+ models including ensembles. Excellent multi-series support.

```
pip install autots
```

PyCaret

Low-code ML library. Time series module with `compare_models()`. Great for EDA + model selection in one workflow.

```
pip install pycaret[time_series]
```

Auto-sklearn

Bayesian optimization for sklearn pipelines.
Strong meta-learning component that transfers from prior experiments.

```
pip install auto-sklearn
```

NeuralForecast

Deep learning-based AutoML for forecasting.
NBEATS, NHITS, TFT architectures with automatic tuning.

```
pip install neuralforecast
```

StatsForecast

Ultra-fast statistical forecasting. AutoARIMA, AutoETS, AutoTheta. 100× faster than statsmodels.

```
pip install statsforecast
```

Evaluation Metrics for Forecasting

Metric	Formula	Units	Best When
MAE	$\text{mean}(y_i - \hat{y}_i)$	Same as target	Outliers need equal weight
RMSE	$\sqrt{\text{mean}((y_i - \hat{y}_i)^2)}$	Same as target	Large errors must be penalised
MAPE	$\text{mean}(y_i - \hat{y}_i / y_i) \times 100$	Percentage	Scale-free comparison needed
SMAPE	$\text{mean}(2 y_i - \hat{y}_i / (y_i + \hat{y}_i))$	Percentage	Avoid division by zero in MAPE
MASE	$\text{MAE} / \text{MAE}(\text{naïve})$	Unitless	Comparing across datasets



AutoML + Feature Hashing

Solving the multi-series scalability problem with hashing-based global models

AutoML + Feature Hashing

Solving the multi-series scalability problem with hashing-based global models

The Multi-Series Scalability Problem

Amazon

SKUs requiring demand forecasting

10M+

Retail Chain

Store × SKU combinations weekly

1M+

Energy Grid

Smart meter time series

100K+

Ad Platform

Campaign performance series

Why Local Models Fail at Scale:

✗ Local Model

Train one ARIMA/Prophet per series.
500K models = days of compute. Memory explosion. Impossible to maintain.

✗ Cold Start

New products / stores have no history.
Local models fail immediately. Requires special handling.

✗ Feature Sharing

Similar series (same category/region) share patterns but local models can't learn this cross-series structure.

Local vs Global Forecasting Models

● Local Models

- One model per time series
- ✓ Fully customized to each series
- ✓ No interference between series
- ✗ Doesn't scale beyond ~1,000 series
- ✗ Cold start problem for new series
- ✗ Cannot learn cross-series patterns
- Examples: ARIMA, Prophet (per series)

● Global Models

- One model trained across ALL series
- ✓ Scales to millions of series
- ✓ Learns shared patterns globally
- ✓ Handles cold start via features
- ✗ Requires rich feature representation
- ✗ Risks series-interference if not handled
- Examples: LightGBM + lag features

Feature Hashing (The Hashing Trick)



Memory Efficient

Fixed output size regardless of cardinality. No growing vocabulary dict.



No Pre-training

New categories handled at inference without re-fitting the feature encoder.



Deterministic

Same input always produces same hash. No randomness in encoding.



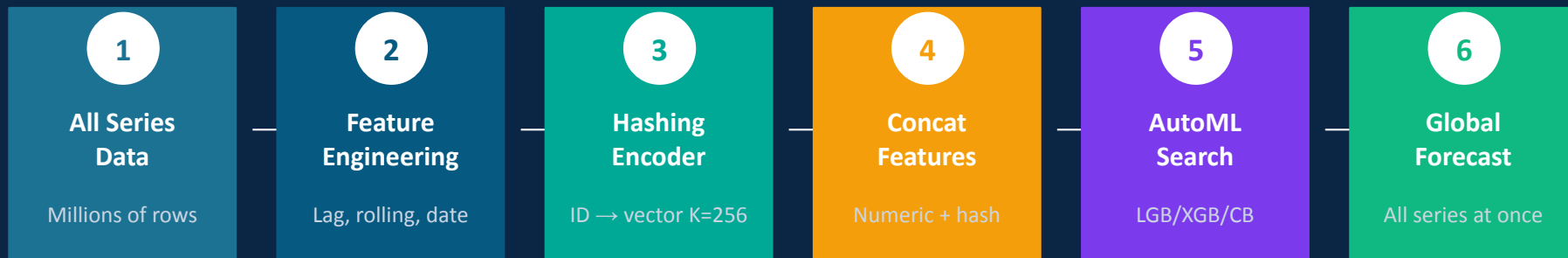
Handles Collisions

Multiple inputs may map to same hash bucket — usually minor performance impact.

Why Feature Hashing for Multi-Series?

- Problem: Store/product IDs are high-cardinality categorical variables (millions of unique values)
- One-hot encoding: 1M categories → 1M columns — memory and compute infeasible
- Label encoding: ordinal structure misleads the model (`store_1` ≠ 'less than' `store_2`)
- Feature hashing: maps any string ID to a fixed-size vector of K dimensions using hash functions
- K is user-defined (e.g. K=128, 256, 512) — controls trade-off between accuracy and memory
- Sklearn: `FeatureHasher(n_features=256, input_type='string')` — supports string inputs natively
- Enables one global LightGBM/XGBoost model for all series — trained once, infers for all
- Combined with lag/rolling features per series → powerful global forecasting architecture

AutoML + Hashing: Full Pipeline



Sample Code

```
from sklearn.feature_extraction import FeatureHasher
from lightgbm import LGBMRegressor
import pandas as pd

# Hash categorical series IDs
hasher = FeatureHasher(n_features=256, input_type='string')
hashed = hasher.transform(df[['store_id', 'item_id']].astype(str).to_dict('records'))

# Combine with lag/rolling features
X = pd.concat([df[lag_cols], pd.DataFrame(hashed.toarray())], axis=1)
model = LGBMRegressor(n_estimators=500).fit(X_train, y_train)
```

Cross-Series Learning with Global Models

- A global model trained on ALL series simultaneously learns shared structure across the dataset
- Series in the same category share seasonal patterns — the global model learns these jointly
- Regularization prevents overfitting to high-variance individual series
- Practically: LightGBM trained on 10M rows (across 100K series) outperforms 100K local models
- Transfer: a new series with zero history gets predictions from global model via its hashed features
- Meta-learning: the model implicitly learns 'what kind of series has what kind of pattern'
- Critical requirement: the global model must receive series identity as a feature (via hashing)
- Result: 100–1000× faster training time vs local approach with competitive accuracy

Comparison & Decision Guide

Choosing the right method for your forecasting problem

Full Method Comparison

Criterion	Auto-ARIMA	Prophet	AutoML	AutoML+Hash
Scalability	Low	Low	Medium	High ✓
Interpretability	High ✓	High ✓	Medium	Low
Handles seasonality	✓ (SARIMA)	✓ ✓ Built-in	Via features	Via features
Handles holidays	Manual	✓ Built-in	Via features	Via features
Multi-series	1 model/series	1 model/series	1 model/series	1 global model ✓
Cold start	✗ Needs history	✗ Needs history	Limited	✓ Via hashing
Exogenous vars	Limited	✓ Regressors	✓ ✓ Native	✓ ✓ Native
Setup complexity	Low	Low	Medium	High
Best for	Univariate, small	Business w/ events	Multi-feature ML	High-cardinality

Decision Guide: Matching Problem to Method

-  1–50 series, classic patterns, linear trend → Auto-ARIMA (SARIMA if seasonal)
-  Business data with known holidays, clear seasonality, stakeholder-friendly → Prophet
-  Many exogenous features, non-linear relationships, medium series count → AutoML (LightGBM/XGBoost)
-  100K+ series, cold-start needed, operational scale → AutoML + Feature Hashing
-  When in doubt: baseline with Auto-ARIMA, then try Prophet, compare with LightGBM AutoML
-  Latency requirements matter: ARIMA fastest inference, neural models slowest
-  Always validate with walk-forward CV — never split randomly for time series
-  Start simple: naive seasonal baseline first, then add complexity only when justified

Industry Use Cases



Retail

- Demand forecasting per SKU & store
- Inventory optimization
- Markdown / promotional planning
- Supply chain scheduling



Technology

- Server capacity planning
- API traffic forecasting
- CI/CD pipeline queue prediction
- User engagement trend analysis



Finance

- Credit risk score trends
- Fraud pattern detection
- Revenue & cost forecasting
- Options pricing (volatility)



Energy

- Electricity demand forecasting
- Renewable generation prediction
- Grid load balancing
- Fuel consumption planning

Key Takeaways

1

Automated forecasting removes the need for manual model selection — machines search the parameter space faster and more consistently than experts

2

Auto-ARIMA excels for univariate statistical forecasting; Prophet excels for business data with seasonality and holidays; choose based on your data characteristics

3

AutoML transforms time series into tabular problems through feature engineering — lag features, rolling statistics, and date features are essential

4

Feature Hashing solves the cardinality problem for global models — enabling one model to forecast millions of series at operational scale

5

Always validate with walk-forward cross-validation and compare against a naive seasonal baseline before declaring a model 'good'

Exercises: Hands-On Practice

01

Auto-ARIMA Exploration

- Load the airline passengers dataset (classic monthly series from 1949–1960)
- Apply `pmdarima.auto_arma()` with `seasonal=True` and `m=12`
- Compare best model AIC against manually selected `SARIMA(1,1,1)(1,1,1)12`
- Plot 12-month forecast with 95% confidence intervals

02

Prophet with Holidays

- Use the M5 Walmart daily sales dataset (or any daily retail dataset)
- Fit Prophet with US holidays and weekly seasonality
- Tune `changepoint_prior_scale` via grid search over `[0.001, 0.01, 0.1, 0.5]`
- Evaluate MAPE on a 28-day holdout set, plot component decomposition

03

Global LightGBM + Hashing

- Simulate 500 time series with different patterns (use random seeds)
- Apply `FeatureHasher(n_features=64)` on series IDs
- Create `lag_1`, `lag_7`, `rolling_7` features for each series
- Train one global LightGBM — compare RMSE vs 500 local ARIMA models

References

- Hyndman & Athanasopoulos — Forecasting: Principles and Practice (3rd ed.) — otexts.com/fpp3
- Taylor & Letham (2018) — Forecasting at Scale — Meta Prophet paper
- pmdarima docs — Auto-ARIMA in Python — alkaline-ml.com/pmdarima
- Microsoft FLAML — fast-automl.readthedocs.io
- Nixtla StatsForecast & NeuralForecast — github.com/Nixtla
- sklearn FeatureHasher — scikit-learn.org/stable/modules/feature_extraction.html